DISTRIBUTED OBJECTS (UNIT 3)

1. Write a note on message passing and distributed objects?

Message Passing

Message passing is a basic communication model in distributed computing where processes communicate by sending and receiving messages over a network. It is a natural model because it resembles human communication and is suitable for network services where processes interact through message exchange. Each message contains marshalled data in an agreed format and is interpreted according to a protocol; receiving a message triggers an action in the receiving process.

Characteristics of Message Passing

1. **Data-oriented paradigm** – focus is mainly on the data contained in messages.
2. **Tightly coupled processes** – communicating processes must remain in direct contact during interaction.
3. **Protocol dependent** – sender and receiver must follow the same message format and protocol.
4. **Suitable for simple services** – such as client-server communication and echo services.

Limitations of Message Passing

- If communication fails due to link or system failure, collaboration stops.
- In complex applications with many requests and responses, interpreting messages becomes difficult.
- Managing communication protocols becomes cumbersome.

Because of these limitations, a higher-level abstraction called **distributed objects** is introduced.

Distributed Objects

The distributed object paradigm provides abstractions beyond message passing. It is based on objects that exist in a distributed system. In object-oriented programming, an object represents an entity and encapsulates:

1. **State (Data)** – stored in instance variables
2. **Operations (Methods)** – used to access or modify the state

A **distributed object** is an object whose methods can be invoked by a remote process running on another computer connected through a network.

Working of Distributed Objects

Instead of sending raw messages, a client process invokes a method on a remote object.

- The method executes on the remote host
- Parameters are passed as arguments
- The result is returned as a return value

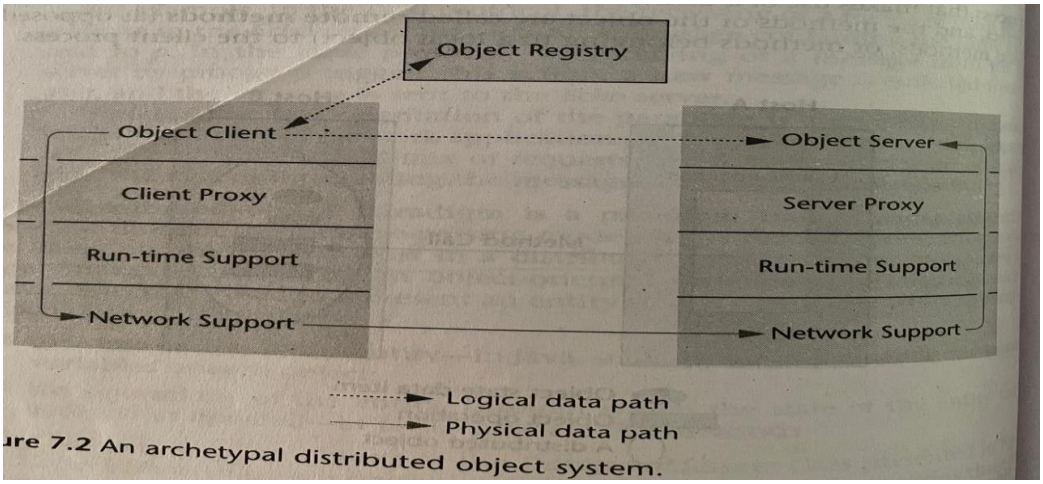
This is called **Remote Method Invocation (RMI)**.

Thus, distributed objects are:

- **Action-oriented** rather than data-oriented
- More suitable for object-oriented software development
- Easier to manage for complex applications

2. **With a block diagram, explain an archetypal distributed object architecture.**

A distributed object system allows an object located on one computer to be accessed by another object or client located on a different computer through remote method invocation (RMI). This architecture hides the complexity of message passing and makes remote communication appear like a normal local method call. The client does not need to know where the object is physically located because the middleware manages the communication between systems.



ure 7.2 An archetypal distributed object system.

The client object is the application that requests a service from a remote object. It invokes a method on the remote object in the same way it would call a local method. For example, if the client needs to perform an addition operation, it simply calls the method without worrying about the network communication involved.

The client proxy, also called the stub, acts as a local representative of the remote object on the client side. When the client makes a method call, the stub receives the request and converts the method parameters into a message format. This process is known as marshalling. The stub then sends the request to the remote system and waits for the response. After receiving the result, it passes it back to the client object.

The distributed object runtime or middleware is the core communication layer of the architecture. It manages message passing between the client and server systems. It provides services such as communication handling, transport management, naming service, object registration, security, and failure handling. Technologies like Java RMI, CORBA, and DCOM are examples of such middleware systems.

The server proxy, also called the skeleton, works on the server side. It receives the request message sent by the client stub and converts it back into method parameters through a process called unmarshalling. After that, it invokes the required method on the actual server object. Once the method execution is completed, the skeleton collects the result and sends it back to the client through the middleware.

The server object is the real object that provides the service requested by the client. It contains both data and methods. The actual business logic is executed here. For example, if the method is an addition function, the computation takes place inside the server object and the final result is returned to the client.

The working of the architecture begins when the client invokes a method on the remote object. The client stub converts the parameters into a message and sends it through the middleware. The middleware transfers the request over the network to the server side. The server skeleton receives the request, converts it back into parameters, and calls the required method in the server object. After execution, the result follows the reverse path and reaches the client as if it were a normal local method return value.

Thus, the archetypal distributed object architecture provides location transparency and communication transparency. It simplifies distributed programming by hiding low-level message passing details and allows developers to build distributed applications using familiar object-oriented concepts.

3. **Write a note on distributed object system.**

A distributed object system is a model in distributed computing in which software objects located on different computers communicate and work together across a network. It extends the concept of object-oriented programming to distributed environments. In this system, an object can invoke methods of another object even if that object is located on a remote machine. This makes remote communication appear similar to a normal local method call.

In object-oriented programming, an object contains two important parts: state and behavior. The state is represented by data members or instance variables, while the behavior is represented by methods or operations. A distributed object system allows these objects to be shared across multiple computers while maintaining the same object-oriented principles.

The main idea of a distributed object system is Remote Method Invocation (RMI). Instead of sending raw messages between processes, a client object directly invokes a method on a remote object. The system automatically handles the communication details such as sending requests, receiving responses, and converting data into transferable formats. This makes programming easier and reduces the complexity of message passing.

Some common examples of distributed object systems are Java RMI, CORBA (Common Object Request Broker Architecture), DCOM (Distributed Component Object Model), and SOAP-based web service systems. These technologies are used in enterprise applications, banking systems, e-commerce platforms, and other large-scale distributed applications.

4.What do you mean by remote procedure call? With diagram, explain local procedure call and remote procedure call.

Remote Procedure Call (RPC)

Remote Procedure Call (RPC) is a communication mechanism in distributed systems that allows a program to execute a procedure or function on another computer in a network as if it were a local procedure call. It hides the details of network communication and makes the remote service appear like a normal function call. The client sends a request to the remote server, the server executes the procedure, and the result is returned to the client.

Local Procedure Call (LPC)

A local procedure call is a normal function call that takes place within the same process or on the same machine. The calling program transfers control directly to the called procedure using the system's runtime environment. Since both procedures are present in the same memory space, execution is fast and simple.

In a local procedure call, parameters are passed directly through the stack or registers, and the result is returned immediately after execution. No network communication is involved.

Remote Procedure Call (RPC)

In a remote procedure call, the called procedure is located on another machine connected through a network. The client calls the procedure as if it were local, but internally the RPC system handles communication between the client and server.

The client stub receives the procedure call and converts the parameters into a message format using marshalling. This message is sent through the network to the server stub. The server stub performs unmarshalling, calls the actual procedure on the server, collects the result, and sends it back to the client. The client receives the result as if it were a normal local call.

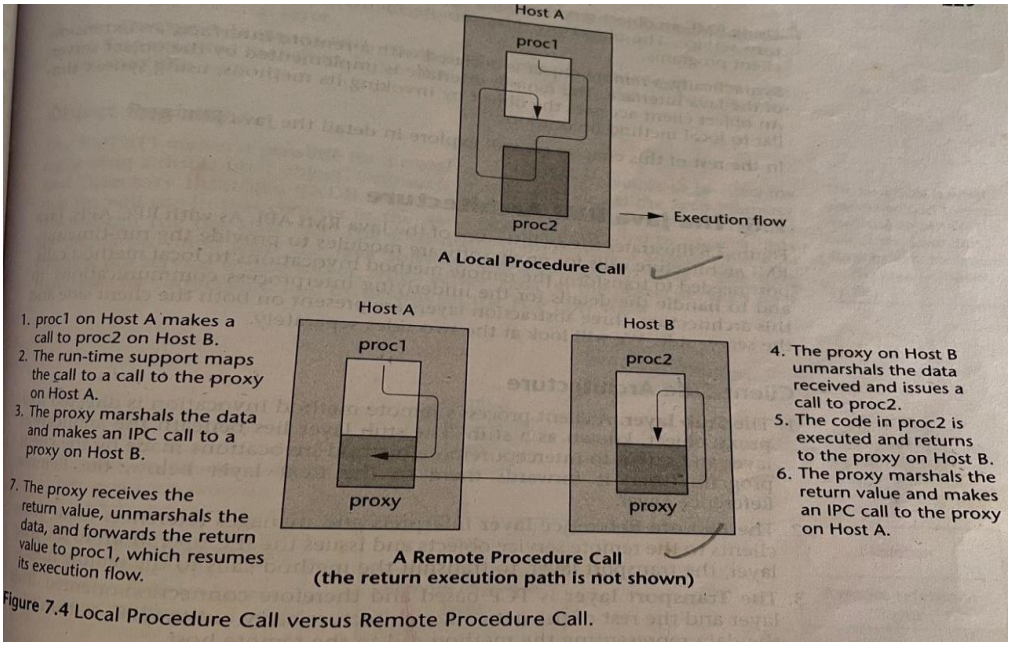


Figure 7.4 Local Procedure Call versus Remote Procedure Call.

Difference between LPC and RPC

Local Procedure Call	Remote Procedure Call
Procedure exists in same system	Procedure exists in remote system
No network communication	Network communication required
Faster execution	Slower due to network delay
Simple parameter passing	Requires marshalling and unmarshalling
No stub needed	Uses client stub and server stub

5. Write a note on RMI.

Remote Method Invocation (RMI) is a mechanism in distributed computing that allows an object running on one computer to invoke the methods of an object located on another computer through a network. It is mainly used in Java-based distributed object systems. RMI makes remote communication appear like a normal local method call, so the programmer does not need to handle low-level network details such as sockets or message passing.

RMI follows the distributed object model where the client object requests services from a remote server object. The client invokes methods on the remote object just like calling a local method. The system automatically manages communication between the client and the server and returns the result after method execution.

The working of RMI begins when the server creates a remote object and registers it with the RMI registry using a unique name. The client looks up this object from the registry and obtains a reference to the stub. When the client invokes a method, the stub converts the method parameters into a message through marshalling and sends it to the server. The skeleton receives the request, performs unmarshalling, invokes the actual method on the remote object, and sends the result back to the client.

However, RMI also has some limitations. It is mainly language-dependent because standard Java RMI works only with Java applications. It may also have performance overhead due to serialization and network communication. Security and network failures must also be managed carefully.

Thus, Remote Method Invocation (RMI) is an important distributed computing technology that enables remote communication using object-oriented concepts. It simplifies the development of distributed applications by allowing remote objects to be accessed as if they were local objects.